

libmr — Framework MapReduce in C

Relazione di progetto — Laboratorio 2 A, A.A. 2025-26

Leonardo Corsi — 656276

06/2026

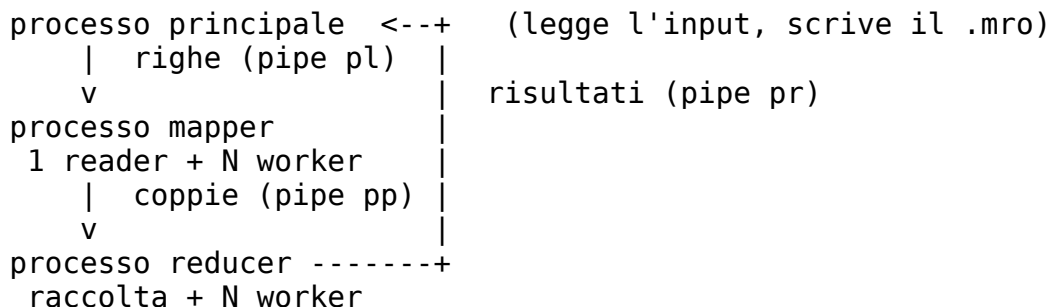
1. Architettura del framework

Libmr è una **libreria statica** (`libmr.a`) con un unico header pubblico `mr.h`.

Il programma utente fornisce due callback — *mapper* e *reducer* — e il framework si occupa di tutto il resto: lettura dell'input, creazione della pipeline, comunicazione, raggruppamento, sincronizzazione e scrittura dell'output. Il framework rimane **generico**: in `src/` non vive alcuna logica applicativa.

La pipeline è una catena di **3 processi** collegati da **3 pipe** unidirezionali.

Mapper e reducer sono processi **multithread C11**:



Il modello di esecuzione segue le tre fasi di MapReduce:

```
file_line -> <token,value>* -> <token,value[]> -> result.
```

La fase di **raggruppamento** è interamente a carico del framework, dentro il processo reducer.

Principio guida e determinismo. Ogni fase produce lo *stesso* output della versione sequenziale di riferimento. La proprietà non è gratuita: con mapper e reducer paralleli l'ordine di arrivo delle coppie è non-deterministico e potrebbe tranelare nell'output. Per impedirlo, i valori di uno stesso token sono ordinati per **contenuto** (`memcmp+size` in `cmp_pair`), non per ordine di arrivo;

allo stesso modo i token sono ordinati lessicograficamente. La conseguenza è che, a parità di input, callback e parametri, il file `.mro` è identico **byte-per-byte** fra esecuzioni diverse e **indipendente dal numero di thread** (1+1 o 8+8): il parallelismo accelera il calcolo ma non si vede nel risultato.

Vale anche al variare della funzione di hash token->reducer, perché il riordino finale assorbe la diversa assegnazione ai thread.

L'identità è byte-per-byte sulla stessa macchina (l'output `.mro` usa l'endianness nativa, scelta lecita per un progetto single-machine). Questa invarianza è verificata dai test ed è usata per distinguere un bug di concorrenza da un bug di logica.

Stato interno e assenza di variabili globali. Tutto lo stato di un'elaborazione risiede in struct `mr` (handle opaco). Non esistono variabili globali o static con stato: questo permette di avere più `mr_t` indipendenti nello stesso processo senza interferenze.

Vincoli tecnologici rispettati. Solo `fork/pipe/dup2/waitpid` e thread C11 (`threads.h`: `thr_create`, `mtx_t`, `cnd_t`). Nessun `pthread`, socket, memoria condivisa o `exec`.

Non si esegue mai `fork()` dopo aver creato thread C11 attivi: i thread nascono **dentro** i processi figli, dopo le `fork`.

2. Interfaccia pubblica (`include/mr.h`)

L'header espone:

- `mr_t` — handle **opaco** (struct `mr` è definita solo in `mr.c`).
- `mr_attr_t` — struttura di configurazione, completamente definita nell'header così da essere una variabile locale dell'utente. Campi: `mapper_threads`, `reducer_threads`, `queue_size`, `log_file`, `hash + hash_arg`.
- `mr_file_line_t` — la riga logica passata al mapper (`file_name`, `file_name_len`, `line_number` da 1, `line`, `line_len`).
- `mr_value_t` — coppia `{const void *data; size_t size;}` per i valori opachi passati al reducer.
- I tipi delle callback: `mr_mapper_t`, `mr_reducer_t`, e le due funzioni di emissione `mr_emit_pair_t` / `mr_emit_result_t`.

Funzioni:

```
int mr_attr_init/destroy(mr_attr_t*);
int mr_attr_set_mapper_threads/reducer_threads/queue_size(mr_attr_t*,
size_t);
int mr_attr_set_log_file(mr_attr_t*, const char *path);
int mr_attr_set_hash_function(mr_attr_t*, mr_hash_t, void *hash_arg);
int mr_create(mr_t*, const mr_attr_t*, mr_mapper_t, mr_reducer_t, void
*user_arg);
int mr_start(mr_t, const char *input_path, const char *output_path);
int mr_destroy(mr_t);
```

Contratto. Tutte le funzioni ritornano 0 (successo) o -1 (errore, con errno impostato). mr_attr_init imposta default validi (≥ 1 thread mapper e reducer); i setter rifiutano valori non validi (0 thread, queue_size 0).

mr_create **copia** la configurazione, dopodiché l'utente può distruggere o riusare l'mr_attr_t. mr_start è **bloccante**.

I puntatori passati alle callback sono validi **solo durante l'invocazione**: il framework copia i dati prima che emit ritorni.

Byte opachi (contratto cruciale). Il token è una C-string alfanumerica, usata *solo* come chiave di raggruppamento (su di esso strlen/strcmp sono leciti).

I **valori** e i **risultati** sono invece sequenze di byte opache di lunghezza nota: il framework li copia con memcpy+size e non assume mai che siano stringhe, interi o terminati da '\0'. Possono contenere byte nulli.

3. Organizzazione dei processi (fork, pipe, dup2, waitpid)

Tutta l'orchestrazione è in mr_start(). La sequenza:

1. **stat** dell'input (distingue file regolare da directory) e apertura del file di output (fopen, "wb").
2. Creazione delle **3 pipe**: pl (righe: padre->mapper), pp (coppie: mapper->reducer), pr (risultati: reducer->padre).
3. **fork del mapper.** Nel figlio: chiusura di tutti i descrittori non usati, dup2(pl[0], STDIN) e dup2(pp[1], STDOUT), chiusura degli originali già duplicati, poi mapper_child(...) e _exit().
4. **fork del reducer.** Nel figlio, simmetricamente: dup2(pp[0], STDIN) e dup2(pr[1], STDOUT), poi reducer_child(...) e _exit().

5. Nel **padre** restano aperti solo i due descrittori che gli servono davvero: `pl[1]` (scrittura verso il mapper) e `pr[0]` (lettura dai risultati). Tutti gli altri vengono chiusi.
6. Il padre **invia le righe** al mapper (`send_file_lines` o `send_dir_lines`).
7. Il padre **chiude `pl[1]`**: questo è il segnale di fine input (EOF) che si propaga a cascata lungo la pipeline.
8. Il padre **legge i risultati** da `pr[0]` e scrive ogni record nel `.mro`.
9. **`waitpid`** su entrambi i figli; il loro exit status concorre al codice di ritorno di `mr_start`.

Chiusura dei descrittori. Un descrittore di scrittura lasciato aperto nel processo sbagliato impedisce l'arrivo dell'EOF e blocca la pipeline. Per questo i descrittori sono inizializzati a -1 e un unico blocco di cleanup (`goto fail`) chiude *solo* quelli effettivamente aperti, sia nel percorso normale sia in quello d'errore.

Niente `exec`. I figli ereditano via `fork` il codice del framework e i puntatori a funzione delle callback dell'utente: `exec` sostituirebbe l'immagine del processo e renderebbe inapplicabile il modello. Per la stessa ragione `user_arg` modificato in un figlio non è visibile altrove.

SIGPIPE ignorato. All'inizio di `mr_start` si installa `SIG_IGN` per SIGPIPE (ripristinato alla fine): una `write` su pipe senza lettore diventa così un errore EPIPE gestibile da `write_n`, non un segnale che terminerebbe il processo.

4. Organizzazione dei thread C11 (mapper e reducer)

I thread nascono **dopo** le `fork`, dentro i figli, e usano esclusivamente `thrd_create` / `mtx_t` / `cnd_t`. Le firme dei thread sono `int f(void *arg)`, compatibili con `thrd_start_t`.

Processo mapper (`mapper_child`) — schema *reader -> coda -> N worker*:

- **1 thread lettore** (`mapper_reader_t`) legge le righe serializzate da `stdin`, ricostruisce una `mr_file_line_t` e la inserisce nella coda condivisa. A EOF marca la coda come *chiusa* e termina.
- **N thread worker** (`mapper_worker_t`) estraggono righe dalla coda, invocano la mapper dell'utente e ne **emettono** le coppie scrivendole sulla pipe verso il reducer. Quando la coda è chiusa e vuota, ogni worker termina.

- Il processo mapper **chiude la pipe verso il reducer una sola volta**, dopo il `thrd_join` di *tutti* i worker (la chiusura avviene implicitamente con `_exit` del figlio, una volta che ogni worker che può produrre coppie è terminato). Così il reducer non riceve EOF prima che le coppie siano scritte.

Processo reducer (`reducer_child`) — schema *raccolta -> raggruppamento -> N worker -> flush ordinato*:

- Una **fase di raccolta** legge tutte le coppie dalla pipe in un `collector_t` in RAM (non serve un thread lettore dedicato: il reduce vero comincia solo a input esaurito, perché la reducer va invocata *una sola volta per token* dopo aver raccolto tutti i valori).
- Si **ordina** e si **raggruppa** per token (§7).
- **N thread worker** (`reducer_worker_t`) elaborano i gruppi: ogni worker prende in carico i gruppi il cui token gli compete secondo l'hash (§7), invoca la reducer e accumula i risultati in un **buffer per-gruppo**.
- Il thread principale del reducer effettua il **flush sequenziale** dei buffer in ordine di token sulla pipe verso il padre (determinismo, §7–8).

5. Code interne e meccanismi di sincronizzazione

Coda produttore-consumatore (`queue_t`). È una struttura **privata al processo** (non una pipe, non memoria condivisa): un **ring buffer** di capacità `queue_size` (`mr_attr_set_queue_size`) protetto da:

- `mtx_t mtx` — mutua esclusione su tutti i campi;
- `cnd_t not_full` — il produttore attende qui se la coda è piena;
- `cnd_t not_empty` — i consumatori attendono qui se la coda è vuota;
- `int closed` — flag di chiusura.

`queue_push` blocca finché c'è spazio (o la coda è chiusa); `queue_pop` blocca finché c'è un elemento o la coda è chiusa-e-vuota (ritorna allora un valore che segnala terminazione). `queue_close` imposta il flag e fa `cnd_broadcast` su entrambe le condition variable per sbloccare tutti i thread in attesa. Questa è la base della **terminazione pulita**: il reader chiude la coda a EOF, i worker svuotano i residui e terminano.

Scrittura sincronizzata sulla pipe. Più worker mapper scrivono coppie sulla stessa pipe verso il reducer. Per garantire che un messaggio logico (header + token + valore) non venga **interlacciato** con quello di un altro thread, ogni worker serializza l'intera coppia sotto un `mtx_t write_mtx` condiviso. Il contesto di emissione (`pipe_emit_t`) porta con sé `fd`, il mutex e un flag `error`.

Pool del reducer. Lo stato condiviso dai worker reducer è `reduce_pool_t` (array dei gruppi, numero di thread, funzione hash, e un flag `error` protetto da `mtx_t`). I worker **non** condividono buffer di output: ciascuno scrive nel buffer del proprio gruppo, quindi l'unica sincronizzazione è il flag d'errore (il primo worker che fallisce lo alza, gli altri si fermano al giro successivo).

6. Formato dei messaggi sulle pipe (protocollo interno)

La comunicazione usa un **protocollo a lunghezze esplicite**, perché valori e risultati possono contenere byte arbitrari (testo libero non basterebbe).

Letture/scritture parziali. Sono implementate `read_n` e `write_n`, che ciclano finché non hanno trasferito esattamente `n` byte (o EOF/errore). Una singola `read/write` su pipe non garantisce il trasferimento di un intero messaggio.

Coppia (mapper -> reducer). Header a lunghezze fisse seguito dai dati:

```
typedef struct { int tok_len; int val_len; } pair_header_t;
/* poi: tok_len byte di token, poi val_len byte di valore opaco */
```

`tok_len` **non** include il `'\0'`: il ricevente alloca `tok_len + 1` byte e aggiunge il terminatore. `val_len` è il numero esatto di byte opachi (nessun terminatore implicito; può essere 0).

Riga (padre -> mapper). Header analogo con `file_name_len`, `line_len`, `line_number`, seguito dal nome file e dal contenuto della riga (senza `'\n'`).

Validazione degli header. Le lunghezze viaggiano come `int`. In ricezione si verifica che **non siano negative** e che non superino limiti documentati prima di convertirle a `size_t`:

- `MR_MAX_TOKEN_L = 4096, MR_MAX_FNAME_L = 4096;`
- `MR_MAX_VALUE_L = MR_MAX_LINE_L = 16 MiB.`

Un header troncato o fuori range produce un errore EPROTO senza dereferenziare lunghezze non validate. **Endianness**: il progetto è single-machine, quindi gli interi sono scritti in formato nativo con `write_n` diretto, senza conversione di rete.

stdout riservato. Nei processi figli `stdout` è la pipe del protocollo: le callback applicative non devono scriverci (per diagnostica si usa `stderr` o il log del framework).

7. Struttura dati per il raggruppamento per token

Il raggruppamento avviene nel reducer, in tre passi:

1. **Raccolta** — tutte le coppie sono lette in un `collector_t`, un array dinamico (raddoppio di capacità) di `pair_t {char *token; void *value; size_t value_size;}`, con token e valore copiati (memoria propria del reducer).
2. **Ordinamento** — `qsort` con il comparatore `cmp_pair`: prima per **token** (`strcmp`), e a parità di token per **contenuto del valore** (`memcmp` sul minimo delle due lunghezze, poi per lunghezza). Ordinare i valori per *contenuto* — e non per ordine di arrivo, non deterministico con mapper paralleli — è ciò che rende l'output riproducibile anche per reducer *sensibili all'ordine*.
3. **Raggruppamento** — una scansione lineare dell'array ordinato individua i blocchi con lo stesso token e costruisce, per ciascuno, un `group_t`: il token, una **view** `mr_value_t[]` sui valori del gruppo (senza ricopiarli) e un buffer di output inizialmente vuoto. La reducer è quindi invocata **esattamente una volta per token**, con tutti i suoi valori.

Assegnazione ai thread (hashing). Ogni worker reducer processa il gruppo `g` solo se `hash(token, len, hash_arg) % reducer_threads == id_worker`. La funzione di default è **FNV-1a** a 64 bit (`mr_default_hash`); l'utente può sostituirla con `mr_attr_set_hash_function`.

La hash è **deterministica** e influenza solo quale thread elabora un token, mai l'output finale.

8. Formato del file di output (.mro)

Il file di output è binario, a **record di lunghezza esplicita**, un record per risultato emesso:

```
[token_len: u32][token: token_len byte][result_len: u32][result:
result_len byte]
```

La lunghezza del token **non** include il '\0'. Interi scritti in formato nativo (single-machine), coerentemente col protocollo delle pipe.

Determinismo dell'ordine. I record sono scritti **ordinati lessicograficamente per token**. L'ordinamento globale è garantito dal fatto che i buffer per-gruppo vengono riversati nel .mro **in ordine di token** dal thread principale del reducer (flush sequenziale), anche se i gruppi sono stati calcolati da worker concorrenti. Se la reducer emette **più risultati per lo stesso token**, questi compaiono nell'**ordine di emissione** (sono accodati nel buffer del gruppo nell'ordine in cui emit viene chiamato).

Il formato è opaco: la correttezza non dipende dal fatto che i risultati siano stringhe. Le utility wc_dump (risultato come int) e mro_dump (risultato come testo) servono solo a *leggere* un .mro in forma umana.

9. Formato del file di log e statistiche

Log (mr.log di default, configurabile con mr_attr_set_log_file). Ogni riga ha il formato:

```
[<sec>.<ms>] pid=<N> <ruolo> <messaggio>
```

con ruolo in {parent, mapper, reducer}. Il log registra almeno: creazione delle pipe, fork di mapper e reducer, avvio/terminazione dei thread, apertura/chiusura dei file di input e output, numero di righe inviate al mapper, numero di coppie prodotte, numero di token distinti, numero di risultati finali, e gli errori (con errno tradotto via strerror_r).

Sincronizzazione del log. Il log è condiviso fra thread e fra processi diversi. La sincronizzazione si appoggia alla semantica POSIX di O_APPEND: il file è aperto con O_WRONLY|O_CREAT|O_APPEND e ogni evento è emesso con **una sola write** di un messaggio sotto PIPE_BUF; la scrittura in append è quindi **atomica** e non servono lock applicativi. strerror_r (versione thread-safe, su buffer locale) evita stato condiviso.

File di statistiche. Separato dal log, con nome derivato da `log_file` (`<log_file>.stats`) e quindi **per-istanza**. Aperto con `O_TRUNC` (fresco a ogni run). Contiene: righe lette, coppie prodotte, token distinti, risultati emessi e il **tempo di esecuzione** (`CLOCK_MONOTONIC`, `elapsed_ms`). Esempio:

```
[...] reducer  pairs=<P> tokens=<T>
[...] parent   lines=<L> results=<R> elapsed_ms=<ms>
```

10. Descrizione dei test (`make test`)

La suite (`tests/run.sh`) compila la libreria e gli esempi, poi esegue per ogni caso wordcount, decodifica il `.mro` con `wc_dump` e confronta con l'atteso (`diff`). Copre:

- **Casi limite su singolo file:** file vuoto, sole righe vuote (`only_newlines`), ultima riga senza `'\n'` (`no_final_newline`), riga molto lunga (`long_line`), token ripetuti (`same_token`), token tutti distinti (`all_unique`), case-sensitivity, terminazioni CRLF.
- **Input a directory:** `dir_multi`, `dir_with_subdir` (ricorsione).
- **Determinismo:** due run su directory -> `.mro` identici (`cmp`), incluso il caso con **reducer sensibile all'ordine** (`MR_CONCAT=1`) a 8+8 thread.
- **Invarianza rispetto ai thread:** stesso input a 1+1 e 8+8 thread -> `.mro` identici, sia per il word count sia per l'esempio non-wordcount.
- **Path d'errore:** input inesistente -> uscita non-zero senza crash.
- **Più istanze, statistiche, hash utente:** due `mr_t` indipendenti (`multi`) con configurazioni diverse -> output identico; produzione del file statistiche; hash utente diversa (`MR_HASH=7`) -> stesso `.mro`.
- **Genericità:** `examples/index.c` (indice dei numeri di riga per token) prova il framework con mapper/reducer **diversi dal word count**, valore opaco `unsigned long` e risultato stringa.

L'esito finale è la stampa di `TEST OK` (o `TEST FALLITO` con `exit != 0`).

11. Note sulle scelte progettuali

- **Gestione errori:** macro CHECK per la validazione argomenti (EINVAL) e wrapper SYS/mr_log_err per le system call, con cleanup centralizzato goto fail. Ogni chiamata di sistema è controllata.
- **Stile:** mr.c è organizzato *bottom-up* (nessuna forward declaration); l'ordine dei descrittori è gestito con inizializzazione a -1 e chiusura idempotente per evitare double-close e deadlock da EOF mancante.
- **Scansione ricorsiva delle directory:** send_dir_lines visita la directory in ordine **lessicografico** (qsort dei nomi) e scende nelle sottodirectory, processando i file regolari; l'ordinamento rende deterministico l'ordine di lettura indipendentemente da readdir.
- **Più mr_t indipendenti:** garantito dall'assenza di stato globale — ogni elaborazione vive nel proprio struct mr, con log e statistiche per-istanza (esempio examples/multi.c).
- **File di statistiche separato:** vedi §9.
- **Funzione di hashing utente:** mr_attr_set_hash_function, con default FNV-1a interno (vedi §7).